

Project03

컴퓨터소프트웨어학부 2022095287 김준서

Setting

- 실행 환경: Ubuntu-22.04

Design

- 기존의 proc 구조체를 조금 변경하여 스레드용으로 사용
- 스레드 시스템 콜은 기존의 프로세스 시스템 콜과 유사하게 작성
- 같은 프로세스의 스레드들은 pgdir을 공유함
 - 스레드끼리 stack 영역은 공유되면 안됨
 - 각 스레드는 다른 스레드와 겹치지 않는 stack 페이지를 할당받아 sp를 그곳으로 설정
- xv6와의 상호작용을 위해 기존 프로세스 관련 메소드, 메모리 관련 유틸 메소드를 일부 수정
 - p->sz 대신 p->master->sz 사용
 - p->master->sz에 접근할 때 ptable.lock 사용
- 하드웨어 지원 함수 없이 Lamport's bakery 알고리즘을 이용해 lock & unlock 구현

Implementation

system call

```
// syscall.h
#define SYS_thread_create 22
#define SYS_thread_exit 23
#define SYS_thread_join 24
```

- sysproc.c: 각각의 시스템 콜에 대해 proc.c의 함수를 적절하게 호출하는 wrapper 함수 추가
- 시스템 콜이 호출될 수 있도록 user.h, usys.S, syscall.c, syscall.h, Makefile 수정

proc structure

```
// proc.h
struct proc {
    /* (...) */
    thread_t tid;           // Thread ID
    int is_master;          // 1: master thread, 0: worker thread
    struct proc *master;    // Master thread
    void *retval;
};
```

- 기존의 `proc` 구조체에 멤버를 추가하여 스레드용으로 사용
- `tid`: 모든 스레드는 unique한 스레드 id를 가짐
 - `pid`와 유사하게, 스레드를 생성할 때마다 전역변수 `nexttid`를 증가시키면서 대입
- `is_master`: 프로세스 생성 메커니즘으로 생성된 master 스레드와, `thread_create`로 생성된 worker 스레드를 구분
 - `allocproc`에서는 1로, `allocthread`에서는 0으로 초기화
 - master 스레드에서는 `thread_exit`가 불가능
- `master`: 프로세스를 대표하는 스레드
 - 모든 프로세스는 정확히 1개의 master 스레드를 가짐
 - 같은 `pid`의 worker 스레드는 동일한 `master` 값을 가짐
- `retval`: 스레드가 `thread_exit`에서 반환한 값을 임시 저장

thread_create function

- `curthread` (현재 스레드)의 `pgdir` 값을 `nt` (새 스레드)의 `pgdir`에 복사
 - 두 스레드는 모든 메모리 영역을 공유하게 됨
 - 독립적인 stack 영역을 만들기 위해, `nt->master->sz` 주소부터 페이지 2개를 추가로 할당받음
 - 하나는 가드 페이지로, 하나는 새 스레드의 stack 영역으로 사용
- `fork`에서 참고한 부분
 - 커널 스택 할당 및 초기화
 - 파일 디스크립터 복사
- `exec`에서 참고한 부분
 - 유저 스택 할당 및 초기화
 - `start_routine`에 전달할 인자를 유저 스택에 저장
 - 트랩 프레임을 통해 새 스레드의 `eip`와 `esp`를 지정

thread_exit function

- worker 스레드만 호출할 수 있음 (master 스레드는 `exit` 사용)
- `exit`에서 참고한 부분
 - 파일 디스크립터 정리
 - `curthread` (현재 스레드)를 채널로 하여 sleep해 있던 스레드들을 wakeup
 - `thread_join`은 특정 `tid`를 가진 스레드를 기다리기 때문에, `exit`과는 wakeup하는 채널이 다름

thread_join function

- 끝나기를 기다리는 tid를 가진 스레드의 주소를 찾음 (taddr)
- 해당 스레드가 ZOMBIE 상태가 될 때까지 대기
- 스레드는 pgdir을 공유하므로, reaping할 때 freevm하면 안됨
- wait에서 참고한 부분
 - sleep을 이용한 wait
 - ZOMBIE가 된 스레드 메모리 해제

xv6와의 상호작용

- 공통
 - p->sz 대신 p->master->sz 사용
 - p->master->sz 에 접근할 때 ptable.lock으로 동기화
- fork: 프로세스를 만들 때, proc 구조체에 추가한 일부 멤버 초기화
- exec: 함수 시작 부분에서, 현재 스레드와 같은 프로세스의 다른 스레드들을 정리
- kill: 처음으로 조건을 만족하는 프로세스를 kill → 조건을 만족하는 모든 스레드를 kill
- sbrk: 위의 공통 지시 외에 별도 수정 필요 없음
- sleep, pipe: 별도 수정 필요 없음
- exit: 현재 프로세스 id를 가진 모든 스레드를 ZOMBIE 상태로 변경

lock, unlock

Lamport's bakery 알고리즘을 구현

- https://en.wikipedia.org/wiki/Lamport%27s_bakery_algorithm
- <https://person.dibris.unige.it/delzanno-giorgio/SO1/AA0607/bakery.htm>

Result

thread

- 빌드 커맨드: ./bui1dxv6.sh
 - 또는 make clean; make; make fs.img
- 실행 커맨드: ./bootxv6.sh
- xv6를 실행한 후, thread_test, thread_exec, thread_exit, thread_kill 실행

```

$ thread_test
Test 1: Basic test
Thread 0 start
Thread 0 end
Thread 1 start
Parent waiting for children...
Thread 1 end
Test 1 passed

Test 2: Fork test
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Thread 1 end
Thread 3 end
Child of thread 4 end
Thread 2 end
Thread 4 end
Test 2 passed

Test 3: Sbrk test
Thread 0 start
Thread 1 start
Thread 2 startThread 3 start
Thread 4 start

Test 3 passed

All tests passed!
$

$ thread_exec
Thread exec test start
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Hello, thread!
$
$ thread_exit
Thread exit test start
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Exiting...
$

$ thread_kill
Thread kill test start
Killing process 14
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
Kill test finished
$

```

lock

- 빌드 및 실행 커맨드: `gcc -o pthread_lock_linux pthread_lock_linux.c -lpthread; ./pthread_lock_linux`
- `#define DEBUG 1`를 주석 해제하면, (1) 작업이 진행 중인지 (2) 총 실행시간이 몇 ms인지 알 수 있다

```

4
5  int shared_resource = 0;
6
7  #define NUM_ITERS 500000
8  #define NUM_THREADS 2000
9
10 void lock(int);
11 void unlock(int);
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 2

```

junseo@junseo:~/ps$ gcc -o pthread_lock_linux pthread_lock_linux.c -lpthread; ./pthread_lock_linux
shared: 1000000000

```

Trouble Shooting

- 스레드를 생성할 때, 유저 스택 영역이 데이터를 공유하지 않게 하려면 어떻게 해야 하는가?
 - pgdir 값 자체(`pde_t*`)를 복사
 - 스레드들이 모든 유저 메모리 영역을 공유하게 됨
 - sp 값을 다른 스레드와 겹치지 않는 va로 초기화
 - ustack을 위한 페이지를 `p->master->sz`에 할당하기 때문에, stack과 heap 영역이 뒤섞이게 됨
 - pgdir가 가리키는 테이블(`pde_t`)을 엔트리별로 전부 복사

- pde 혹은 pte 가 갱신될 경우, 스레드들 간의 동기화 문제 발생
- ex) sbrk에 의해 pte 가 해제되었다가 재할당되는 경우

→ 스레드 생성 속도와 구현 복잡도를 고려하여 a번으로 결정

2. thread_test.c 의 Sbrk test에서 xv6가 재부팅되는 문제 발생

- 프로세스의 크기는 p->master->sz 로 대표성 있게 관리하고자 했음
- p->master->sz 가 예상되는 값과 달라 이상한 메모리를 참조하게 됨
 - ptable.lock을 사용하여 동기화 문제 해결
 - sysproc.c 의 sys_sbrk, syscall.c 의 fetchint 등 p->sz 에 접근하는 xv6 함수를 더 발견하여 고침

3. wait, thread_join는 ZOMBIE 상태인 프로세스, 스레드를 reaping한다. 둘을 구분해야 하는가? 어떻게 구분하는가?

- pthread에서 테스트해본 결과, 둘을 구분함
 - wait는 thread_exit을 reaping하지 않음
 - exit이 발생하면 thread_join하던 스레드도 같이 종료됨 (둘은 같은 pid를 가질 것이므로)
- is_master 멤버를 통해 구분할 수 있음
 - wait에서는 !p->is_master 이면 continue
 - thread_join에서는 p->is_master 이면 continue

4. 스레드 공유 영역 동기화를 위한 lock에서, 스레드 개수가 늘어날수록 실행시간이 급격히 늘어나고 불균등해짐

- 진행 상황을 로깅해보면 데드락 문제는 아님
- 스레드가 많아질수록 busy waiting의 비율이 높아지는 것을 병목 원인으로 판단
- busy waiting을 위한 루프를 돌 때마다 usleep을 호출하여 불필요한 반복을 제거함

→ 실행시간이 줄어들고 균일해짐